Integrated Sampler-Adaptive Learning Benchmark

Code Guide and Design Notes

for `benchmark_sal_integrated_ml_physics_qpu_vs_classical.py`

This document explains, in detail, how the new benchmark code works. The code combines sampler-adaptive learning ideas from the 2025 SAL paper with the previous machine-learning benchmark and the earlier generalized transverse-field Ising-model code. The result is one integrated program that can train RBMs, masked RBMs, and semi-restricted Boltzmann machines using exact, Gibbs, simulated-annealing, Langevin-SB, Pegasus-QPU, or Zephyr-QPU negative-phase samplers, while also supporting physics-derived datasets from long-range 1D TFIM, frustrated J1-J2 TFIM, and 2D Edwards-Anderson spin glass models.

Main benchmark script	`benchmark_sal_integrated_ml_physics_qpu_vs_classical.py`
QPU helper	`qpu_runtime_safe_sampler_extended.py`
Primary training framework	SAL-style positive phase + sampler-specific negative phase
Extra physics metric	Variational energy of $\psi_\theta(s) = \sqrt{Q_\theta(s)}$

1 What this program is trying to do

The benchmark has two goals.

First, it compares several sampling backends inside the *same* Boltzmann-machine training loop:

- exact marginalization,
- Gibbs sampling,
- simulated annealing,
- Langevin simulated bifurcation (LSB),
- a Pegasus-targeted D-Wave QPU backend,
- a Zephyr-targeted D-Wave QPU backend.

Second, it evaluates those samplers on both standard machine-learning datasets and physics-derived datasets. For the physics-derived case, the code does not stop at generative metrics such as KL divergence. It also interprets the trained Boltzmann machine as a positive wavefunction and reports the corresponding variational energy for the chosen Hamiltonian. This makes the script useful for benchmarking both "how well does the model learn the data distribution?" and "how physically good is the learned positive-amplitude state?"

1.1 Why the SAL paper matters here

The SAL paper contributes two ideas that are central to this code:

1. a fast non-MCMC sampler, Langevin simulated bifurcation (LSB), and
2. conditional expectation matching (CEM), which estimates an effective inverse temperature β_{eff} for samplers whose output temperature is not known a priori.

The new code generalizes those ideas beyond the exact RBM/SRBM setup of the paper in three directions:

- it adds the `masked_rbm` architecture,
- it allows non-LSB negative-phase samplers such as simulated annealing and real D-Wave QPUs to reuse the same CEM temperature-estimation machinery,
- it connects data-driven BM training to physics targets built from transverse-field Ising Hamiltonians.

2 Files and their responsibilities

The implementation is split into two files.

2.1 Main script

`benchmark_sal_integrated_ml_physics_qpu_vs_classical.py` is the orchestrator. It owns:

- dataset generation,
- physical Hamiltonian generation,
- model construction,
- all training backends except the low-level Ocean glue,
- CEM fitting,
- the training loop,
- plotting,
- multiprocessing and output management.

2.2 QPU helper

`qpu_runtime_safe_sampler_extended.py` isolates Ocean- and embedding-specific logic. It owns:

- solver selection by topology or explicit solver name,
- embedding search and caching,
- support for disconnected and bias-only logical graphs,
- safe read-budget calculation with `estimate_qpu_access_time()`,
- repeated batched submissions under the runtime limit,
- return of expanded samples in a deterministic variable order.

The "disconnected-graph" improvement is especially important for SAL/CEM, because the conditional hidden-only problems used by CEM often contain no couplers at all; they are just collections of independent hidden spins with shifted biases.

3 Mathematical model used by the code

3.1 Energy and score conventions

The code parameterizes a Boltzmann machine with visible spins $v \in \{-1, +1\}^{N_v}$ and hidden spins $h \in \{-1, +1\}^{N_h}$ through visible biases a, hidden biases b, hidden-visible couplings W, and optional visible-visible couplings L.

Its unnormalized joint *score* is

$$S(v, h) = a \cdot v + b \cdot h + h^T W v + \frac{1}{2} v^T L v. \tag{1}$$

This is the sign-convention twin of the paper's energy $E(v, h) = -S(v, h)$. Training therefore uses a gradient-ascent style *positive minus negative* update, whereas the paper writes the same physics as gradient descent on the energy-form KL objective.

3.2 Visible marginal at finite effective inverse temperature

Given an effective inverse temperature β, the visible marginal is

$$Q_\beta(v) \propto \sum_h e^{\beta S(v,h)} = \exp\left[\beta\left(a \cdot v + \frac{1}{2} v^T L v\right) + \sum_{j=1}^{N_h} \log 2\cosh(\beta[b_j + (Wv)_j])\right]. \tag{2}$$

This expression is implemented by the function `visible_log_unnorm_beta()`. A key design choice is that the code uses this exact finite-β visible marginal everywhere it matters: in the exact backend, in exact KL evaluation, in pseudo-log-likelihood evaluation, and in the physics-side variational-energy calculation.

3.3 Model families

The same parameterization supports three architectures.

RBM	Dense hidden-visible bipartite couplings, no visible-visible edges.
Masked RBM	Same as RBM, but the hidden-visible matrix W is sparsified by a random mask.
SRBM	Dense hidden-visible couplings plus a visible-visible mask L. The visible mask can be ring, dense, grid, or the interaction graph of the selected physics problem.

4 Datasets and physical targets

4.1 Machine-learning style datasets

The code implements four purely data-distribution benchmarks.

- **Bars and stripes**: all valid binary bar or stripe patterns on a small grid.
- **Parity**: all visible states with fixed parity.
- **Planted Ising**: a classical pairwise Ising distribution on a ring.
- **Three-spin**: an exact Boltzmann distribution of a random three-body spin-glass Hamiltonian, included because it mirrors the SAL paper's first synthetic rugged-landscape benchmark.

Each of these generators returns

- the sampled training data,
- the exact support states when feasible,
- the exact target probabilities on that support.

That exact support is what makes it possible to evaluate KL divergence exactly for small systems.

4.2 Physics-derived dataset: `physics_ground`

For the physics mode, the code first builds a stoquastic transverse-field Ising Hamiltonian and then computes the exact ground-state distribution in the computational basis,

$$P_D(s) = |\psi_0(s)|^2. \tag{3}$$

The supported Hamiltonians are:

$$H_{\text{long}} = -\sum_{i<j} \frac{1}{r_{ij}^{\alpha}} \sigma_i^z \sigma_j^z - \Gamma \sum_i \sigma_i^x, \tag{4}$$

$$H_{J_1 J_2} = -J_1 \sum_i \sigma_i^z \sigma_{i+1}^z - J_2 \sum_i \sigma_i^z \sigma_{i+2}^z - \Gamma \sum_i \sigma_i^x, \tag{5}$$

$$H_{\text{EA}} = -\sum_{\langle ij \rangle} J_{ij} \sigma_i^z \sigma_j^z - \Gamma \sum_i \sigma_i^x, \tag{6}$$

with $J_{ij} = \pm 1$ in the Edwards-Anderson case.

The exact ground-state probabilities are obtained by sparse diagonalization of the Hamiltonian matrix, using `eigsh()` on the stoquastic sparse operator.

4.3 Why this physics mode is useful

This design lets the code ask two distinct questions.

1. Can a trained BM reproduce the exact ground-state probability distribution in the σ^z basis?
2. If that BM is converted into a positive wavefunction, is it also good in variational-energy terms?

This is the bridge between "energy-based generative modeling" and "neural ansatz for many-body physics."

5 How the backends are implemented

5.1 Exact backend

The exact backend enumerates all visible states and evaluates the finite-β visible marginal exactly. It then computes

- $\langle v_i \rangle$,
- $\langle h_j \rangle$,
- $\langle h_j v_i \rangle$,
- $\langle v_i v_j \rangle$,

by marginalization.

This backend is the "gold standard" negative phase whenever N_v is small enough that the support size 2^{N_v} remains manageable.

5.2 Gibbs backend

The Gibbs backend stores persistent visible and hidden chains. Each negative phase performs:

1. a blocked hidden update, because hidden units are conditionally independent given visible units,
2. a sequential visible update, which is required when the model contains visible-visible couplings.

For the RBM case, this reduces to the standard alternating blocked Gibbs sampler. For the SRBM case, it becomes a proper semi-restricted sequential visible update.

5.3 Simulated-annealing backend

The simulated-annealing backend also stores persistent chains, but instead of sampling from one fixed conditional distribution, it runs a geometric cooling schedule from `sa_temp_start / beta_submit` down to `sa_temp_end / beta_submit`. This makes its output distribution only an *effective* Boltzmann sampler, not an exact finite-temperature Markov chain. That is exactly why CEM is useful: the positive phase should use the effective β_{eff} inferred from the actual sampler output, not the nominal cooling schedule.

5.4 LSB backend

The LSB backend is the direct algorithmic import from the SAL paper. It evolves positions and momenta with

$$Y \leftarrow Y + \Delta\,(J\,\mathrm{sgn}(X) + f), \tag{7}$$

$$X \leftarrow \mathrm{sgn}(X) + \Delta Y, \tag{8}$$

and after each step discretizes the positions and reinitializes the momenta from a Gaussian distribution. In code this is implemented as:

1. sample initial $S \in \{-1, +1\}^{N_v+N_h}$ and Gaussian Y,
2. compute the force from the full BM graph,
3. update Y and then X,
4. discretize back to spins,
5. repeat for `lsb_steps` iterations.

The important conceptual point is that LSB does *not* have a known target inverse temperature. The code therefore always treats LSB as a backend requiring CEM unless the user overrides that behavior with a fixed temperature.

5.5 QPU backends

The Pegasus and Zephyr backends use the same main ideas as the earlier QPU benchmark code.

- The BM is converted to an Ising model whose minimized energy is $-S(v, h)$.

- A fixed embedding is cached and reused.
- Complete bipartite RBMs use a biclique embedding fast path.
- SRBMs and other masked graphs fall back to generic minor-embedding.
- The helper estimates QPU access time before submission and splits the logical sample budget into safe child submissions.

The extended helper adds one more feature that the earlier code did not have: if the conditional CEM problem has no couplers, the helper assigns isolated logical variables to unused physical qubits one by one. This makes hidden-only conditional sampling possible on the real QPU as well.

6 Conditional expectation matching (CEM)

6.1 Why CEM is needed

For exact Gibbs sampling at a chosen β_{submit}, the effective inverse temperature is known. For simulated annealing, LSB, and real QPU sampling, the output distribution is only approximately Boltzmann and the actual effective temperature can drift with model parameters and sampler settings.

The positive phase, however, needs hidden expectations of the form

$$\langle h_j \rangle_{v,\beta} = \tanh\Big(\beta\,[b_j + (Wv)_j]\Big). \tag{9}$$

If the wrong β is used here, the positive phase becomes inconsistent with the negative phase. CEM fixes that mismatch.

6.2 How CEM is implemented in the code

The function `estimate_beta_eff()` performs the following steps.

1. Choose one or more conditioning vectors r from the training data, or from random visible states if the user requests that.
2. Build the hidden-only conditional problem with fields
$$f_j(r) = b_j + (Wr)_j. \tag{10}$$
3. Ask the current backend to generate conditional hidden samples under the same sampler settings used in the full negative phase.
4. Compute empirical hidden means from those samples.
5. Fit β_{eff} by minimizing
$$\frac{1}{KN_h}\sum_{k=1}^{K}\sum_{j=1}^{N_h}\Big[m_{k,j}^{\text{sample}} - \tanh\left(\beta f_j(r_k)\right)\Big]^2. \tag{11}$$

The minimization is one-dimensional and uses `scipy.optimize.minimize_scalar()` on a bounded interval.

6.3 Estimator modes

The user can select three modes.

- `auto`: exact and Gibbs use the known submitted inverse temperature, all other samplers use CEM.
- `cem`: always run CEM.
- `fixed`: do not estimate anything; always use `fixed_beta_eff`.

The `auto` mode is generally the best default because it is faithful to the SAL logic without paying unnecessary QPU cost when the temperature is known analytically.

7 Training loop, step by step

The heart of the program is `run_single_experiment()`. Each epoch runs in the following order.

1. Estimate or reuse β_{eff}.
2. Select a minibatch if the user asked for minibatching; otherwise use the full dataset.
3. Compute the positive-phase statistics using analytic hidden means at the current β_{eff}.
4. Generate negative-phase samples with the chosen backend.
5. Update parameters by positive minus negative moments, with L_2 regularization and clipping.
6. Optionally evaluate pseudo-log-likelihood, reconstruction error, exact KL divergence, and physics variational energy.
7. Save a detailed row in `history.csv`.

In compact form, the update is

$$a \leftarrow a + \eta(\langle v \rangle_D - \langle v \rangle_{\text{neg}}) - \lambda a, \tag{12}$$
$$b \leftarrow b + \eta(\langle h \rangle_D - \langle h \rangle_{\text{neg}}) - \lambda b, \tag{13}$$
$$W \leftarrow W + \eta(\langle hv \rangle_D - \langle hv \rangle_{\text{neg}}) - \lambda W, \tag{14}$$
$$L \leftarrow L + \eta(\langle vv \rangle_D - \langle vv \rangle_{\text{neg}}) - \lambda L, \tag{15}$$

where the data-side hidden expectations use β_{eff} and the negative-phase moments come from the backend samples themselves.

8 Why the physics-side variational energy is valid

For a physics-derived dataset, the trained BM defines a visible probability model $Q_\theta(s)$. The code then constructs the positive wavefunction

$$\psi_\theta(s) = \sqrt{Q_\theta(s)}. \tag{16}$$

For the stoquastic transverse-field Ising models considered here, the local energy for a basis state s is

$$E_{\mathrm{loc}}(s) = E_z(s) - \Gamma \sum_i \frac{\psi_\theta(s^{(i)})}{\psi_\theta(s)}, \tag{17}$$

where $E_z(s)$ is the diagonal Ising contribution and $s^{(i)}$ is the state with spin i flipped.

Because

$$\log \psi_\theta(s) = \frac{1}{2} \log Q_\theta(s), \tag{18}$$

these ratios can be evaluated exactly from the BM visible log probabilities. That is what the function `exact_physics_variational_energy_per_spin()` does.

This is one of the strongest features of the code: a purely generative training benchmark can be turned into a many-body physics benchmark without introducing a second, unrelated ansatz implementation.

9 QPU runtime safety and cost control

9.1 Why a dedicated helper is used

Direct QPU use is expensive and failure-prone if runtime is not controlled. The helper therefore performs the following procedure automatically.

1. Determine or reuse a fixed embedding.
2. Count the physical qubits required by that embedding.
3. Query the selected solver for an estimated QPU access time.
4. Binary-search the largest safe number of reads per child call under the chosen fraction of the solver runtime limit.
5. Split the requested logical sample count across multiple child submissions.

This is the same strategy that was already used in the earlier QPU benchmark, but the new helper adds support for disconnected conditional CEM problems.

9.2 Practical levers for reducing QPU time

The code exposes several knobs that materially affect cost.

- `negative_samples`: the main cost driver.
- `cem_samples`: the cost per CEM call.
- `cem_period`: estimate β_{eff} every k epochs instead of every epoch.
- `num_spin_reversal_transforms`: affects batching overhead.
- `max_runtime_fraction`: determines how conservatively the code stays below the solver runtime limit.

For real QPU studies, it is usually wise to start with small values such as `negative_samples=128`, `cem_samples=32`, and `cem_period=5` while debugging.

10 Multiprocessing and output layout

The top-level `main()` function builds one independent task per (model, backend) pair. With `-processes > 1`, those tasks are dispatched through a spawn-based multiprocessing pool. This keeps QPU and classical runs isolated and avoids shared-state surprises.

The output directory structure is

```
output_dir/
  run_config.json
  dataset_info.json
  all_summaries.json
  figure_final_summary.png
  figure_<model>_training.png
  <dataset_name>/
    <model>/
      <backend>/
        history.csv
        summary.json
        final_params.npz
        best_params.npz
```

The most useful file for post-analysis is usually `history.csv`, because it contains the epoch-by-epoch trace of

- β_{eff},
- KL divergence,
- pseudo-log-likelihood,
- reconstruction error,
- variational energy if applicable,
- QPU timing and embedding diagnostics when relevant.

11 Recommended usage patterns

11.1 Paper-style SAL sanity check

Use the three-spin dataset with small visible dimension and compare exact, Gibbs, SA, and LSB:

```
python benchmark_sal_integrated_ml_physics_qpu_vs_classical.py \
  --models rbm masked_rbm srbm \
  --samplers exact gibbs sa lsb \
  --dataset-source three_spin \
  --n-visible 10 \
  --n-hidden 5 \
```

```
  --epochs 100 \
  --negative-samples 512 \
  --output-dir runs_three_spin
```

11.2 Physics benchmark with exact references

```
python benchmark_sal_integrated_ml_physics_qpu_vs_classical.py \
  --models rbm srbm \
  --samplers exact gibbs sa \
  --dataset-source physics_ground \
  --physical-hamiltonian j1j2_1d \
  --j1j2-size 10 --j1 1.0 --j2 0.5 \
  --visible-graph physics \
  --transverse-field 1.0 \
  --epochs 80 \
  --negative-samples 512 \
  --output-dir runs_j1j2
```

Using `visible_graph=physics` is a particularly sensible ablation here, because it tests whether giving the SRBM the same graph as the Hamiltonian helps relative to a generic ring or dense choice.

11.3 Conservative first QPU run

```
python benchmark_sal_integrated_ml_physics_qpu_vs_classical.py \
  --models rbm srbm \
  --samplers sa qpu_pegasus qpu_zephyr \
  --dataset-source physics_ground \
  --physical-hamiltonian long_range_1d \
  --long-range-size 8 \
  --transverse-field 0.8 \
  --epochs 30 \
  --negative-samples 128 \
  --cem-samples 32 \
  --cem-period 5 \
  --annealing-time 20 \
  --num-spin-reversal-transforms 2 \
  --output-dir runs_qpu_start
```

This is intentionally conservative. Once the pipeline behaves well, raise the sample counts only for the final runs that go into plots.

12 Extension points

The code was written to be easy to extend.

12.1 Add a new sampler

To add another backend, implement a new subclass of `BackendBase` with three methods:

- `negative_phase()`,
- `conditional_hidden_means()`,
- optionally `known_beta_eff()` if the sampler's temperature is analytically known.

This is enough to plug the new sampler into the same SAL training loop.

12.2 Add a new physics Hamiltonian

To add another stoquastic model, define a new builder that returns a `PhysicalProblem` containing

- the number of sites,
- the edge list,
- the coupling values,
- any longitudinal fields.

Then register it in `build_physical_problem()`.

12.3 Add a new dataset

To add a new classical target distribution, register it in `build_dataset()`. If it is small enough to enumerate exactly, provide support states and exact probabilities. Doing so unlocks exact KL evaluation and exact model observable computation automatically.

13 Most important implementation choices

If someone reads only one section of this guide, it should be this one.

1. The code uses the exact finite-β BM visible marginal, not a crude post-hoc division of the visible score by temperature.
2. CEM is backend-agnostic: the same one-dimensional fit is reused for SA, LSB, and QPU backends.
3. Physics targets are integrated at the dataset level, which means the same BM code can be scored by both generative and variational metrics.
4. The QPU helper is robust to disconnected conditional problems, which is essential for practical SAL-style temperature estimation.
5. Multiprocessing is done across full experiments rather than inside the sampler, which keeps the logic simple and avoids hidden shared state.

14 Short reference map of the main functions

Function or class	Role
`build_dataset`	Creates the selected dataset and its exact support metadata.
`build_physical_problem`	Constructs the chosen TFIM-style Hamiltonian graph.
`visible_log_unnorm_beta`	Exact visible log unnormalized probability at inverse temperature β.
`positive_phase_stats`	Data-side statistics with analytic hidden means at β_{eff}.
`ExactBackend`	Exact negative-phase marginalization.
`GibbsBackend`	Alternating blocked/sequential Gibbs sampler.
`SimulatedAnnealingBackend`	Cooling-schedule backend with unknown effective temperature.
`LangevinSBBackend`	LSB implementation inspired by the SAL paper.
`QPUBackend`	Pegasus or Zephyr backend using the runtime-safe helper.
`estimate_beta_eff`	Auto / CEM / fixed effective-temperature management.
`exact_physics_variational_energy_per_spin`	Positive-phase energy evaluator for physics datasets.
`run_single_experiment`	One complete training run for one model-backend pair.
`main`	Task generation, multiprocessing, figure generation, and overall orchestration.

15 Closing remarks

This code is not just a direct port of the SAL paper, and it is not just a minor revision of the earlier benchmark. It is a synthesis of both. The main conceptual novelty is that one can now benchmark classical samplers, optimization samplers, and real D-Wave QPUs inside one temperature-aware BM training loop, and then score the result both as a generative model and, when a physical target is used, as a positive variational state.

That is exactly the kind of structure you want when the real question is not "which sampler wins on one number?" but rather "how do sampler quality, effective temperature, embedding overhead, and physical-state fidelity interact across model classes and hardware topologies?"